# CM5: From Data Analysis to Machine Learning

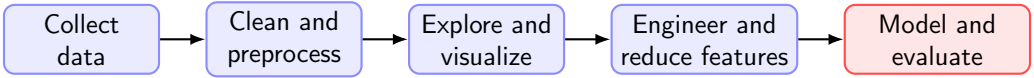
3TLDE.A13 - Législation Numérique et Enjeux de l'Analyse des Données


June 2, 2026

# Table of contents


1. From data analysis to machine learning
2. What is machine learning?
3. The machine learning pipeline
4. K-Nearest Neighbors
5. Summary

# Where are we in the data analysis workflow?




In the previous lectures, we prepared data for analysis:
**cleaning, EDA, feature engineering, and dimension reduction.**

Today, we ask the next question:

**Main question**

Once the data is ready, how can we build a model that makes **predictions on new observations?**

# Learning objectives of CM5

At the end of this lecture, you should be able to:

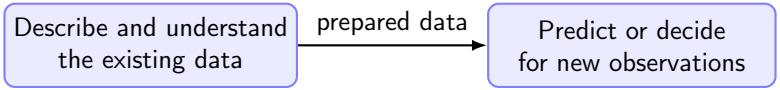
- explain the difference between data analysis and machine learning;
- describe the main steps of a supervised machine learning pipeline;
- distinguish classification and regression tasks;
- explain why train/test split is necessary;
- identify common sources of data leakage;
- explain the intuition of K-Nearest Neighbors;
- apply KNN to classification and regression;
- explain why scaling and dimensionality matter for distance-based models.

# Table of contents

1. From data analysis to machine learning
2. What is machine learning?
3. The machine learning pipeline
4. K-Nearest Neighbors
5. Summary

# From describing data to predicting with data

Data analysis and machine learning are connected, but they do not have exactly the same goal.




## Data analysis

- What is in the data?
- What patterns do we observe?
- Which variables are related?

## Machine learning

- Can we learn from examples?
- Can we predict unseen cases?
- How well does the model generalize?

# The bridge between the previous lectures and today

The previous steps are not independent from machine learning. They directly influence model quality.

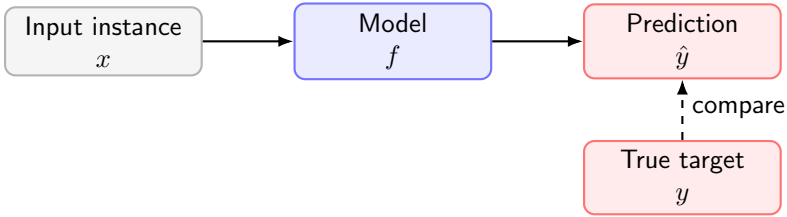
| Previous topic | Role before modeling | Risk if ignored |
|---|---|---|
| Data cleaning | reliable values and types | biased or invalid model |
| EDA | understanding patterns | blind modeling |
| Feature engineering | better representation | weak predictive signal |
| Dimension reduction | compact representation | noisy or sparse features |

**Key idea**

Machine learning does not replace data analysis. It depends on data analysis.

# A prediction problem

In supervised machine learning, we start from examples with known answers.




$$\hat{y} = f(x)$$

**Objective**

Learn a rule from past examples that can make accurate predictions for future examples.

# Table of contents

1 From data analysis to machine learning

2 What is machine learning?

3 The machine learning pipeline

4 K-Nearest Neighbors

5 Summary

# What does it mean for a machine to learn?

A simple definition:

**Machine learning**

A machine learning algorithm uses data to learn a model that improves its performance on a task.

More concretely, in this course:

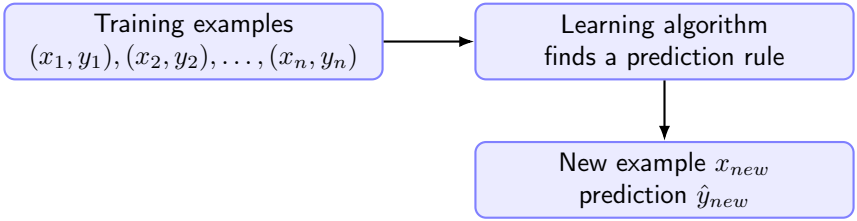
$$\text{training data} \longrightarrow \text{learned prediction rule (model)} \longrightarrow \text{prediction on new data}$$

**Important distinction**

A program written by hand follows explicit rules. A machine learning model derives its rule from examples.

# Supervised learning

In **supervised learning**, each training example comes with a known target.




**Notation**

$x_i$ is an input observation. $y_i$ is the known target. The model predicts $\hat{y}$.

# Supervised learning: classification and regression

The nature of the target variable determines the type of supervised task.

| Task | Target | Prediction | Example |
|---|---|---|---|
| Classification | categorical | class label | species, cancellation, disease |
| Regression | numerical | real value | price, temperature, duration |

**Classification:** $y \in \{\text{class } 1, \ldots, \text{class C}\}$ **Regression:** $\hat{y} \in \mathbb{R}$

# Unsupervised learning

In unsupervised learning, the data has no target label $y$.

$$X \longrightarrow \text{discover structure}$$

## Main idea

- no predefined output;
- learn patterns from features;
- useful for data exploration.

## Examples

- clustering: find groups of similar samples;
- PCA: represent data in fewer dimensions;
- association rules: discover frequent patterns.

**Connection with this course**

EDA helps us understand data manually; unsupervised learning helps us discover structure automatically.

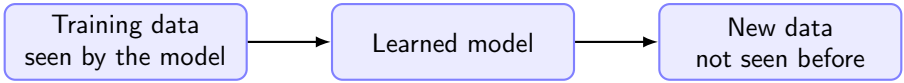
# Model, parameters, and hyperparameters

Three terms are often confused.

| Term | Meaning | Example |
|---|---|---|
| Model | prediction rule used after learning | linear model, tree, KNN |
| Learning algorithm | procedure used to build the model | least squares, tree construction, KNN storage |
| Parameters | learned from data by the algorithm | coefficients in linear regression |
| Hyperparameters | chosen before learning | number of neighbors $K$ |

# Generalization: the central objective

A model should not only perform well on the data it has already seen.



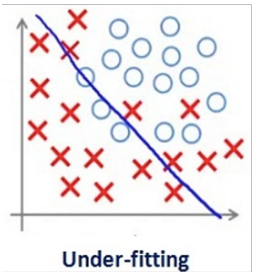

**Generalization**

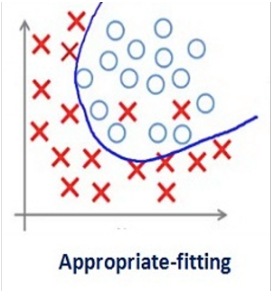
Generalization is the ability of a model to make good predictions on new observations.

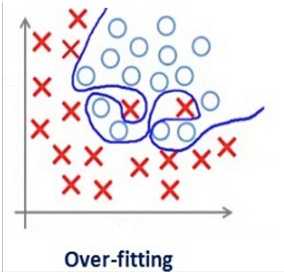
Good machine learning is not memorization. It is prediction beyond the training examples.

# Underfitting and overfitting

- Underfitting: poor training and test performance.
- Appropriate-fitting: good test performance.
- Over-fitting: good training but weak test performance.



# Table of contents

1 From data analysis to machine learning

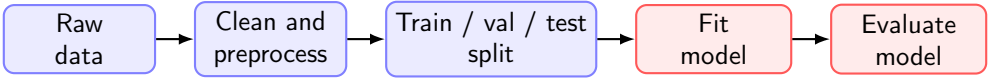
2 What is machine learning?

3 The machine learning pipeline

4 K-Nearest Neighbors

5 Summary

# A standard supervised learning pipeline

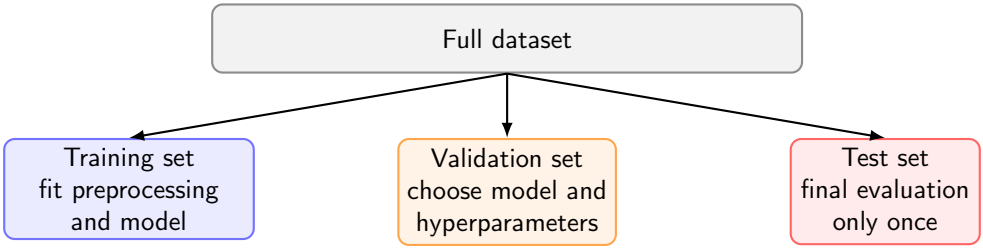



**Practical message**

The machine learning pipeline is not only the algorithm.
It includes data preparation, splitting, transformations, training, evaluation, and interpretation.

# Train/validation/test split

To evaluate generalization, we separate the data before training and model selection.




**Rule**

The test set must simulate **future unseen data**.
It should not influence preprocessing choices, model fitting, or hyperparameter tuning.

# Why do we need a validation set?

The validation set is used to make choices before the final test evaluation.

## Typical choices

- Which features should be used?
- Which preprocessing strategy is better?
- Which model should be kept?
- Which values of the parameters are better?

## Important distinction

- validation: compare alternatives;
- test: estimate final performance;
- the test set is not a tuning set.

**Key message**

If we repeatedly evaluate many choices on the test set, the test performance becomes overly optimistic.

# Creating train/validation/test splits

A first step is to keep an untouched test set, then create a validation set from the remaining data.

```python
from sklearn.model_selection import train_test_split

random_seed = 42
# First split: keep 20% of the data for final testing
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=random_seed,
    stratify=y
)

# Second split: 25% of the remaining 80% becomes validation
# Final proportions: 60% train, 20% validation, 20% test
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val,
    test_size=0.25,
    random_state=random_seed,
    stratify=y_train_val
)
```

# Data leakage

**Data leakage** happens when information from the validation set, test set, or future data enters the training process.

## Wrong

- scale the full dataset;
- impute using all rows;
- perform PCA before splitting;
- choose hyperparameters using the test set;
- use a variable unavailable at prediction time.

## Correct

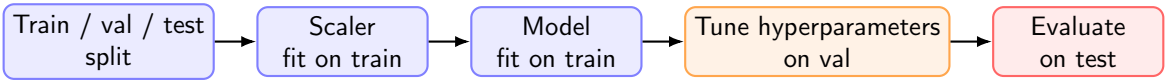
- split first;
- fit preprocessing on training data;
- choose hyperparameters on validation data;
- evaluate once on test data;
- keep only variables available when predicting.

**Key message**

Leakage gives unrealistic performance.
It is one of the most common mistakes in beginner machine learning projects.

# Preprocessing may belong inside the pipeline

A reliable workflow treats preprocessing, fitting, and model selection as one pipeline.




**Important**

Every transformation used on validation or test data must be learned only from training data. The test set is kept untouched until the end.

# Evaluation metrics for classification: accuracy

For classification, the prediction is a class label.

## Accuracy

$$\text{Accuracy} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbb{1}(\hat{y}_i = y_i)$$

Accuracy measures the proportion of correctly classified test samples.

**Example**

If a model correctly classifies 85 samples out of 100, then its accuracy is:

$$\text{Accuracy} = \frac{85}{100} = 0.85$$

**Limitation**

Accuracy can be misleading when classes are imbalanced.

# When accuracy is not enough

Consider a binary classification problem with imbalanced classes.

| Class | Number of samples | Proportion |
|---|---|---|
| Negative | 950 | 95% |
| Positive | 50 | 5% |

A trivial model predicts every sample as negative.

$$\text{Accuracy} = \frac{950}{1000} = 95\%$$

**Problem**

The accuracy is high, but the model detects no positive sample.

We need more detailed metrics based on the types of errors.

# Confusion matrix

A confusion matrix shows how predictions are distributed across true classes.

| | Pred. Positive | Pred. Negative |
|---|---|---|
| True Positive | TP | FN |
| True Negative | FP | TN |

## Correct predictions

- TP: positive correctly predicted as positive;
- TN: negative correctly predicted as negative.

## Errors

- FP: negative incorrectly predicted as positive;
- FN: positive incorrectly predicted as negative.

**Key idea**

The confusion matrix tells us not only how many errors the model makes, but also what kinds of errors it makes.

# Precision, recall, and F1-score

From the confusion matrix, we can define more informative metrics.

| Metric | Formula | Meaning |
|---|---|---|
| Precision | $\frac{TP}{TP+FP}$ | Among predicted positives, how many are correct? |
| Recall | $\frac{TP}{TP+FN}$ | Among true positives, how many are detected? |
| F1-score | $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ | Balance between precision and recall |

**Practical message**

When classes are imbalanced, accuracy should usually be complemented with precision, recall, F1-score, and the confusion matrix.

# Evaluation metrics for regression

For regression, the prediction is a numerical value.

| Metric | Formula | Interpretation |
|---|---|---|
| MAE | $\frac{1}{n}\sum_i \lvert y_i - \hat{y}_i \rvert$ | average absolute error |
| MSE | $\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2$ | mean squared error |
| RMSE | $\sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$ | error in target units |
| $R^2$ | $1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$ | variance explained |

**Practical message**

Always choose an evaluation metric that matches the prediction task and the practical cost of errors.

# Table of contents

1 From data analysis to machine learning

2 What is machine learning?

3 The machine learning pipeline

4 K-Nearest Neighbors

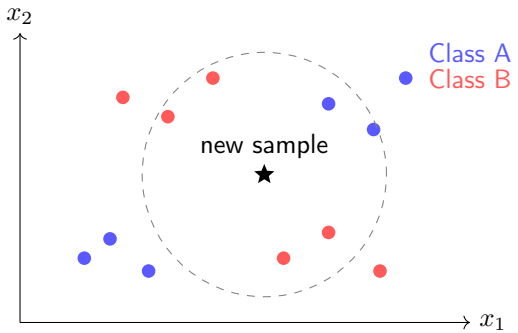
5 Summary

# KNN: learning from neighbors

K-Nearest Neighbors is based on a simple idea:

**K-Nearest Neighbors is based on a simple idea:**

To predict a new observation, find the $K$ most similar training observations and use their known targets.



# KNN is a lazy learning method

Unlike many models, KNN has no explicit parameter-learning stage.

| Model | During `fit` | During `predict` |
|---|---|---|
| Linear regression | learn coefficients | apply formula |
| Decision tree | build a tree | follow tree rules |
| Neural network | learn weights | forward pass |
| KNN | store training examples | compute distances to neighbors |

**Key message**

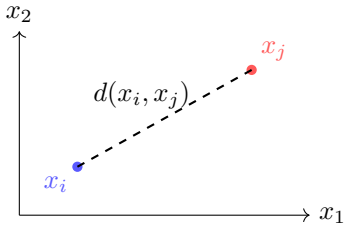
For KNN, the main computation happens at prediction time.
This is why KNN is called a **lazy learner** or **instance-based method**.

# Distance between samples

KNN needs a way to measure similarity between observations.

For numerical features, a common choice is Euclidean distance:

$$d(x_i, x_j) = \sqrt{\sum_{p=1}^{d} (x_{ip} - x_{jp})^2}.$$




**Consequence**

Because KNN is distance-based, feature scaling is essential.
Otherwise, variables with large numerical scales can dominate the distance.

# Choosing the number of neighbors $K$

The hyperparameter $K$ controls the locality of the prediction.

**Small $K$**

- very local decisions;
- sensitive to noise;
- can overfit;
- complex decision boundary.

**Large $K$**

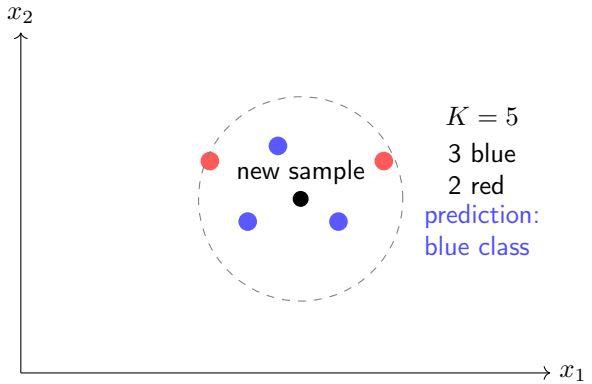
- smoother decisions;
- less sensitive to noise;
- can underfit;
- more global behavior.

**Practical rule**

$K$ is usually selected by validation or cross-validation, not by guessing.

# KNN classification: majority vote

For classification, KNN predicts the most frequent class among the $K$ nearest neighbors.



# Mathematical view of KNN classification

Let $\mathcal{N}_K(x)$ be the set of the $K$ nearest training samples to $x$.

KNN classification predicts:

$$\hat{y} = \arg\max_c \sum_{i \in \mathcal{N}_K(x)} \mathbb{1}(y_i = c).$$

**Interpretation**

For each class, count how many neighbors belong to this class. The predicted class is the one with the largest count.

This is simple, but powerful enough to produce nonlinear decision boundaries.

# Uniform vote or distance-weighted vote

Neighbors can contribute equally or according to their distance.

## Uniform weights

**Each neighbor has weight 1**

- simple majority vote;
- all neighbors have the same influence.

## Distance weights

**Closer neighbors have larger weights**

- often uses weights proportional to $1/d$;
- can reduce the effect of far neighbors.

**In scikit-learn**

`KNeighborsClassifier(weights="uniform")` or `weights="distance"`.

# Choosing $K$ with a validation set

Assume that the data has already been split into training, validation, and test sets.

```python
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np

candidate_k = [1, 3, 5, 10, 15, 20, 30]
validation_scores = []

for k in candidate_k:
    clf = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])

    clf.fit(X_train, y_train)
    y_val_pred = clf.predict(X_val)

    val_acc = accuracy_score(y_val, y_val_pred)
    validation_scores.append(val_acc)

best_index = np.argmax(validation_scores)
best_k = candidate_k[best_index]

print("Best K:", best_k)
```

# Final evaluation on the test set

After choosing $K$, we train the final model and evaluate it once on the test set.

```
X_train_final = np.concatenate([X_train, X_val], axis=0)
y_train_final = np.concatenate([y_train, y_val], axis=0)

final_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=best_k))
])

final_clf.fit(X_train_final, y_train_final)

y_test_pred = final_clf.predict(X_test)

print("Test accuracy:", accuracy_score(y_test, y_test_pred))
print(confusion_matrix(y_test, y_test_pred))
```

**Important rule**

The test set is used only once, after all choices such as $K$ have been fixed.

# KNN regression: average of neighbors

For regression, the target is numerical.

KNN predicts the average target value among the $K$ nearest neighbors:

$$\hat{y} = \frac{1}{K} \sum_{i \in \mathcal{N}_K(x)} y_i.$$

| Neighbor | Target value |
|---|---|
| 1 | 200 |
| 2 | 220 |
| 3 | 210 |

$$\hat{y} = \frac{200+220+210}{3} = 210$$

# Effect of $K$ in regression

The value of $K$ controls the smoothness of the regression function.

**Small $K$**

- very local prediction;
- can follow noise;
- lower bias but higher variance.

**Large $K$**

- smoother prediction;
- more stable;
- higher bias but lower variance.

**Bias-variance intuition**

KNN gives a simple way to introduce the trade-off between flexibility and stability.

# KNN regressor in scikit-learn

Assume that the data has already been split into training, validation, and test sets. Best value of $K$ has been selected with the validation set.

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

final_reg = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=best_k))
])

final_reg.fit(X_train, y_train)
y_pred = final_reg.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred, squared=False))
print("R2:", r2_score(y_test, y_pred))
```

# Why scaling is critical for KNN

KNN uses distances. Variables with large numerical scales can **dominate** the distance.

| Feature | Typical range | Impact without scaling |
|---|---|---|
| Age | 18 to 90 | small contribution |
| Income | 20,000 to 100,000 | dominates distance |
| Number of rooms | 1 to 10 | almost ignored |

**Practical message**

For KNN, standardization or normalization is usually necessary before computing distances.

# KNN and high-dimensional data

KNN becomes less effective when the number of features is large.

## Problem

- distances become less informative;
- many points appear similarly far away;
- irrelevant features add noise;
- prediction becomes slower.

## Possible solutions

- remove irrelevant features;
- scale features properly;
- use feature selection;
- use PCA or other reductions.

**Connection with CM4**

Feature selection and dimension reduction are especially important for distance-based methods.

# KNN: strengths and limitations

| Strengths | Limitations |
|---|---|
| simple and intuitive | prediction can be slow |
| works for classification and regression | sensitive to feature scaling |
| nonlinear behavior without explicit equations | affected by irrelevant features |
| few assumptions about data distribution | struggles in high dimensions |
| useful baseline model | choice of $K$ matters |

**Takeaway**

KNN is an excellent first machine learning model because it makes the role of preprocessing, distance, and validation very visible.

# Practical checklist before using KNN

Before applying KNN, ask the following questions.

1. What is the prediction target?
2. Is it a classification or regression task?
3. Which features are available at prediction time?
4. Have missing values and categorical variables been handled?
5. Are numerical features scaled?
6. Is the train/validation/test split done before preprocessing?
7. Which values of $K$ should be compared?
8. Which evaluation metric matches the task?

# Table of contents

1 From data analysis to machine learning

2 What is machine learning?

3 The machine learning pipeline

4 K-Nearest Neighbors

5 **Summary**

# Key takeaways

1. Machine learning uses data to learn prediction rules for new observations.
2. Supervised learning uses examples with known targets.
3. Classification predicts categories; regression predicts numerical values.
4. A reliable pipeline includes splitting, preprocessing, fitting, validation, and final evaluation.
5. Data leakage must be avoided by fitting preprocessing and model only on training data.
6. KNN predicts from the targets of the nearest training examples.
7. KNN has little explicit training: it mainly stores the training data.
8. Scaling and dimensionality are crucial for distance-based models.

# Transition to the machine learning course

This lecture introduced machine learning from a data analysis perspective.

The next machine learning course will go further:

- more model families: linear models, trees, ensembles, probabilistic models;
- training objectives and loss functions;
- model selection and cross-validation;
- regularization and overfitting control;
- deeper evaluation and interpretation of models.

**Final message**

Good machine learning begins before the model: it begins with clean data, careful exploration, meaningful features, and a reliable evaluation protocol.